

No Phase Runtime Policy

适用范围：整个 Plush Toy ERP 项目

目标：防止历史 phase 概念继续污染运行时代码、API、菜单、测试和 Codex 判断。

核心原则：Phase 只能作为历史记录，不能作为产品结构、代码结构、任务结构或接口结构。

1. 背景

项目早期使用过类似下面的阶段命名：

```
Phase 7
Phase 8
Phase 9
Phase 10
Phase 11
Phase 12
```

这种命名可以帮助早期规划，但在项目进入产品化内测候选阶段后，phase 命名会带来严重副作用。

典型问题包括：

```
API 被命名成 phase8.createShipment
前端路由变成 /erp/phase8/facts
后端 usecase 变成 Phase8Usecase
repository 变成 phase8_repo.go
测试围绕 phase 编写
Codex 读取代码时误判 phase 是正式业务边界
```

这会导致 AI、开发者、测试和客户都被错误引导。

2. 总体结论

本项目后续不再按 phase 组织开发。

正确开发单位是：

```
Capability
Design Spec
Test Case
```

```
Acceptance Criteria
Delivery Package
```

错误开发单位是：

```
Phase
```

Phase 可以保留在历史文档中，但不得进入运行时结构。

3. 为什么不能按 phase 做

3.1 Phase 会误导 Codex

Codex 读代码时会根据命名推断项目结构。

如果代码中存在：

```
Phase8Usecase
jsonrpc_phase8.go
phase8_repo.go
/erp/phase8/facts
phase8.createShipment
```

Codex 很容易认为：

```
phase8 是一个正式领域
phase8 是一个 API domain
phase8 是一个页面模块
phase8 是一个产品能力边界
```

然后继续在 phase8 里添加新逻辑。

这会让项目越来越偏离真实业务领域。

3.2 Phase 会掩盖真实领域

真实业务领域应该是：

```
sales
purchase
inventory
production
```

```
outsourcing  
shipment  
finance  
bom  
sku  
quality
```

而不是：

```
phase8  
phase9  
phase10
```

客户和实施人员不会理解 phase8，他们只理解：

```
销售订单  
采购订单  
库存预留  
出货单  
生产任务  
财务事实
```

3.3 Phase 会鼓励“阶段完成感”

AI 很容易围绕 phase 生成：

```
phase evidence  
phase report  
phase closure  
phase roadmap  
phase simulated completion
```

但 ERP 真正需要的是：

```
库存不能错  
出货不能重复扣  
采购不能超收  
BOM 状态不能乱  
权限不能绕过  
导入不能重复加库存  
部署不能带 secret
```

这些必须通过功能级测试保证，而不是 phase 文档保证。

3.4 Phase 会污染 API 和测试

如果 API 是：

```
phase8.createFinanceFact
```

测试就会写成：

```
TestPhase8CreateFinanceFact
```

以后迁移到正式领域时，测试、API、页面、权限都要重命名。

这会造成大量无价值重构。

4. Phase 的唯一合法用途

Phase 只允许作为历史记录存在。

允许出现的目录：

```
docs/archive/  
docs/history/  
docs/evidence/
```

允许示例：

```
docs/archive/phase8-evidence.md  
docs/history/phase9-summary.md  
docs/evidence/phase10-simulation-report.md
```

前提：

```
只描述历史。  
不作为当前开发入口。  
不作为产品菜单。  
不作为 API 说明。  
不作为客户交付说明。  
不作为测试入口。
```

5. Phase 禁止出现的位置

5.1 后端运行时代码

禁止：

```
server/internal/**/phase*.go  
server/internal/**/Phase*.go  
server/internal/**/phase8*  
server/internal/**/jsonrpc_phase8.go  
server/internal/**/phase8_repo.go  
server/internal/**/Phase8Usecase
```

禁止 API method：

```
phase8.*  
phase9.*  
phase10.*
```

5.2 前端运行时代码

禁止：

```
web/src/**/Phase8*.jsx  
web/src/**/phase8*  
web/src/**/Phase*.jsx  
/erp/phase8/*  
/erp/phase9/*
```

禁止菜单：

```
Phase 8  
Phase 9  
Phase Facts  
Phase Closure
```

5.3 配置和部署

禁止：

```
config/**/phase8*  
deploy/**/phase8*  
deployments/**/phase8*  
docker compose 中出现 phase8  
.env 中出现 PHASE8
```

部署资料包必须使用业务领域语言：

```
shipment  
inventory  
finance  
production  
outsourcing
```

5.4 测试

禁止：

```
TestPhase8*  
phase8.test.*  
phase8.spec.*
```

测试必须按业务能力命名：

```
TestShipment_Ship_DoesNotDoubleDeduct  
TestInventoryReservation_ConcurrentCannotOverReserve  
TestFinanceFact_SourceIdempotency  
TestProductionFact_CreateValidFact
```

5.5 客户文档

禁止客户可见文档出现：

```
Phase 8 Facts  
Phase 9 Closure  
Phase 10 Mobile Tasks
```

客户文档必须使用业务名称：

出货单
库存预留
生产任务
外协任务
财务业务事实
移动任务

6. 推荐替代命名

6.1 API domain 替代

Phase 命名	业务命名
<code>phase8.createProductionFact</code>	<code>production.createFact</code>
<code>phase8.createOutsourcingFact</code>	<code>outsourcing.createFact</code>
<code>phase8.createShipment</code>	<code>shipment.create</code>
<code>phase8.addShipmentItem</code>	<code>shipment.addItem</code>
<code>phase8.shipShipment</code>	<code>shipment.ship</code>
<code>phase8.cancelShippedShipment</code>	<code>shipment.cancelShipped</code>
<code>phase8.createReservation</code>	<code>inventory.reserve</code>
<code>phase8.cancelReservation</code>	<code>inventory.cancelReservation</code>
<code>phase8.createFinanceFact</code>	<code>finance.createFact</code>

6.2 Go 文件替代

Phase 文件	业务文件
<code>biz/phase8.go</code>	<code>biz/shipment.go</code> , <code>biz/stock_reservation.go</code> , <code>biz/finance_fact.go</code>
<code>data/phase8_repo.go</code>	<code>data/shipment_repo.go</code> , <code>data/stock_reservation_repo.go</code> , <code>data/finance_fact_repo.go</code>
<code>data/jsonrpc_phase8.go</code>	<code>service/jsonrpc/shipment.go</code> , <code>service/jsonrpc/inventory.go</code> , <code>service/jsonrpc/finance.go</code>
<code>Phase8Usecase</code>	<code>ShipmentUsecase</code> , <code>StockReservationUsecase</code> , <code>FinanceFactUsecase</code>

6.3 Frontend 替代

Phase 页面	业务页面
<code>/erp/phase8/facts</code>	<code>/erp/shipments</code>
<code>/erp/phase8/production</code>	<code>/erp/production</code>
<code>/erp/phase8/outsourcing</code>	<code>/erp/outsourcing</code>
<code>/erp/phase8/finance</code>	<code>/erp/finance/facts</code>
<code>Phase8FactsPage.jsx</code>	<code>ShipmentPage.jsx</code> , <code>FinanceFactsPage.jsx</code> , <code>ProductionPage.jsx</code>

6.4 测试替代

Phase 测试	业务测试
<code>TestPhase8Shipment</code>	<code>TestShipment_Ship</code>
<code>TestPhase8FinanceFact</code>	<code>TestFinanceFact_CreateFromSource</code>
<code>phase8.spec.ts</code>	<code>shipment.spec.ts</code>
<code>phase8-facts.test.jsx</code>	<code>finance-facts.test.jsx</code>

7. 新的任务组织方式

以后所有任务按以下格式组织：

CAP-XXX : Capability
TEST-XXX : Test Case
PAGE-XXX : Page
API-XXX : API Contract
MIG-XXX : Migration
DEPLOY-XXX : Deployment

示例：

CAP-SHIP-001 : 出货单
TEST-SHIP-001 : 重复 ship 不重复扣库存
TEST-SHIP-002 : 重复 cancel shipped 不重复回滚
API-SHIP-001 : `shipment.create` / `shipment.ship` / `shipment.cancelShipped`

PAGE-SHIP-001：出货单列表和详情页
MIG-SHIP-001：shipment_items 唯一约束

8. Capability 模板

每个能力应该有独立文档：

docs/product/capabilities/CAP-SHIP-001-shipment.md

模板：

```
# CAP-SHIP-001：出货单

## 1. 目标

## 2. 用户角色

## 3. 页面

## 4. 字段

## 5. 状态机

## 6. API

## 7. 数据表

## 8. 业务规则

## 9. 权限

## 10. 审计

## 11. 测试用例

## 12. 验收标准

## 13. 不做什么
```

9. Test Case 模板

每个测试用例应该清楚说明业务事实。

示例：

```
# TEST-SHIP-001: 重复 ship 不重复扣库存
```

```
## 目标
```

同一个出货单重复确认发货时，不允许重复扣减库存。

```
## 前置数据
```

- SKU: SKU-PLUSH-BEAR-PINK-S
- initial inventory: 20
- shipment qty: 10

```
## 操作
```

1. 调用 shipment.ship
2. 再次调用 shipment.ship

```
## 预期
```

- shipment.status = SHIPPED
- inventory balance = 10
- inventory_txns 只有一条 SHIPMENT_OUT
- sales_order_item.shipped_qty = 10
- finance_facts 只有一条应收事实

10. Codex 工作规则

给 Codex 的任务必须避免：

```
实现 Phase 13  
继续下一阶段  
完善 phase8  
补齐 phase closure
```

应该写成：

请只完成 TEST-SHIP-001：重复 ship 不重复扣库存。

要求：

1. 先添加失败测试。
2. 只修改 shipment 相关 usecase/repo/test。
3. 不允许新增 phase API。
4. 不允许新增 phase 路由。

5. 不允许写 business_records。
6. 不允许绕过 biz。
7. 运行 go test。
8. 输出修改文件和测试结果。

11. 自动化限制规则

11.1 no-runtime-phase 测试

建议新增：

```
scripts/qa/no-runtime-phase.test.mjs
```

扫描路径：

```
server/internal/  
web/src/  
web/scripts/  
config/  
deploy/  
deployments/
```

禁止 pattern：

```
phase8  
phase9  
phase10  
phase11  
phase12  
Phase8  
Phase9  
PHASE8  
PHASE9  
/erp/phase  
jsonrpc_phase
```

允许路径：

```
docs/archive/  
docs/history/  
docs/evidence/
```

11.2 示例测试规则

伪逻辑：

```
for each file in runtimePaths:
  if content contains phase pattern:
    fail

for each file in allowedArchivePaths:
  allow
```

失败信息必须提示：

```
Runtime phase naming is forbidden.
Use business domain names such as shipment, inventory, finance, production,
outsourcing.
```

11.3 加入 CI

该测试必须加入：

```
node --test scripts/**/*.test.mjs
bash scripts/project-scan.sh --strict
bash scripts/qa/strict.sh
```

12. 迁移计划

12.1 第一步：冻结新增

立即执行：

```
禁止新增 phase runtime code
禁止新增 phase API
禁止新增 phase route
禁止新增 phase test
禁止新增 phase menu
```

先加：

```
AGENTS.md
docs/architecture/NO_PHASE_RUNTIME_POLICY.md
scripts/qa/no-runtime-phase.test.mjs
```

12.2 第二步：重命名现有 phase runtime

将现有 phase runtime 改成业务领域。

优先级：

1. JSON-RPC domain
2. Go usecase 名称
3. repository 文件名
4. 前端 route
5. 菜单
6. 测试名
7. 文档引用

12.3 第三步：迁移 docs

将历史 phase 文档移动到：

```
docs/archive/
docs/history/
docs/evidence/
```

产品文档改用：

```
capability
menu
test case
deployment
architecture
```

12.4 第四步：删除兼容 alias

不要保留：

```
phase8.createShipment -> shipment.ship 的 alias  
/erp/phase8/facts -> /erp/shipments 的 redirect
```

当前仍在开发阶段，直接破坏兼容更干净。

13. 人工 Review 清单

每个 PR 必须检查：

- ☐ 是否新增了 phase 命名。
- ☐ 是否新增了 phase API。
- ☐ 是否新增了 phase 前端路由。
- ☐ 是否新增了 phase 测试。
- ☐ 是否把 phase 放进客户文档。
- ☐ 是否保留了 phase alias。
- ☐ 是否使用正式业务领域命名。
- ☐ 是否围绕 capability/test case 实现。
- ☐ 是否有测试证明业务事实正确。

14. 完成定义

本规则落地后，应满足：

- ☐ 根目录 AGENTS.md 存在。
- ☐ NO_PHASE_RUNTIME_POLICY.md 存在。
- ☐ no-runtime-phase 自动化测试存在。
- ☐ CI 会执行 no-runtime-phase 测试。
- ☐ server/internal 中无 phase runtime 命名。
- ☐ web/src 中无 phase route/page 命名。
- ☐ API method 中无 phase domain。
- ☐ 客户文档中无 phase 产品命名。
- ☐ phase 只存在于 docs/archive、docs/history、docs/evidence。
- ☐ 新任务按 CAP/TEST/PAGE/API/MIG/DEPLOY 组织。

15. Codex 任务拆分

NOPHASE-01：添加 AGENTS.md

请完成 NOPHASE-01：添加根目录 AGENTS.md。

要求：

1. 添加项目级 Codex 规则。
2. 明确禁止 runtime phase 命名。
3. 明确开发单位是 Capability + Test Case, 不是 Phase。
4. 明确禁止 business_records 写正式事实。
5. 明确 backend layering。
6. 明确测试和安全要求。

NOPHASE-02：添加 NO_PHASE_RUNTIME_POLICY.md

请完成 NOPHASE-02：添加 docs/architecture/NO_PHASE_RUNTIME_POLICY.md。

要求：

1. 解释为什么 phase 只能作为历史记录。
2. 列出禁止出现 phase 的目录。
3. 列出允许出现 phase 的 archive 目录。
4. 给出 phase -> domain 命名映射。
5. 给出 Codex 任务规则。

NOPHASE-03：添加 no-runtime-phase 自动化测试

请完成 NOPHASE-03：添加 no-runtime-phase 自动化测试。

要求：

1. 新增 scripts/qa/no-runtime-phase.test.mjs。
2. 扫描 server/internal、web/src、web/scripts、config、deploy、deployments。
3. 禁止 phase8/phase9/Phase8/PHASE8/jsonrpc_phase 等 pattern。
4. docs/archive、docs/history、docs/evidence 允许。
5. 加入 node --test scripts/**/*.test.mjs。

NOPHASE-04：迁移现有 phase runtime 命名

请完成 NOPHASE-04：迁移现有 phase runtime 命名。

要求：

1. 将 phase8 JSON-RPC domain 拆成 production、outsourcing、shipment、inventory、finance。
2. 重命名 Phase8Usecase 和相关 repo。
3. 重命名前端 /erp/phase8 路由。
4. 不保留兼容 alias。
5. 更新测试。
6. no-runtime-phase 测试必须通过。

16. 最终原则

一句话：

Phase 只记录历史，Capability 才指导开发，Test Case 才证明完成。

更具体：

不要让 AI 继续围绕 phase 补代码。
不要让 phase 出现在运行时。
不要让 phase 成为 API、页面、测试或客户语言。
所有新工作必须绑定 capability、业务事实源和自动化测试。